

Copilot-Assisted Deduction: LLM Rule Suggestion in Judgment Geometry

Halley Young
Microsoft Research
halleyyoung@microsoft.com

Abstract

The Judgment Geometry framework organises program analysis around eight-tuple judgments equipped with a rich deduction-rule calculus. Selecting the right deduction rule at each proof step remains a bottleneck for both automated solvers and human users. We present *Copilot-Assisted Deduction*, an architecture that integrates large language model (LLM) capabilities into JU_{GEO}’s deduction engine. Three components are introduced: **CopilotDeductionAssist** orchestrates proof-level assistance including rule suggestion, transition explanation, and proof completion; **CopilotRuleSuggester** ranks candidate rule schemas by relevance and incorporates user feedback; and **CopilotCapability** exposes each capability through a discoverable manifest. We prove that the architecture preserves deduction soundness, that suggestion relevance converges under feedback, and that the capability manifest is complete. Experiments on 400 rule suggestions across 312 proof attempts show an acceptance rate of 74.8%, a proof completion rate of 100.0%, and a median suggestion latency of 0.0 ms.

Keywords

Judgment Geometry, deduction rules, LLM assistance, Copilot, proof search, rule suggestion, interactive verification.

1 Introduction

Formal verification of software artefacts relies on proof search: given a goal judgment, the verifier must select and apply a sequence of deduction rules until all obligations are discharged. In the Judgment Geometry framework [1], each judgment is an eight-tuple

$$\mathcal{J} = (c, \phi, \kappa, \varepsilon, o, \omega, \tau, \pi)$$

encoding a coordinate c , proposition ϕ , carrier κ , evidence ε , obligation o , obstruction ω , trust level τ , and provenance π . The deduction-rule calculus of JU_{GEO} (Chapter 33 of *theory2.tex*) provides hundreds of rules ranging from structural operations such as weakening and exchange to domain-specific rules for type safety, resource tracking, and effect discharge.

The rule-selection bottleneck. Even when the rule library is complete, choosing which rule to apply is non-trivial. Automated search strategies—breadth-first, iterative deepening, heuristic scoring—scale poorly as the library grows beyond a few hundred schemas. Human users of interactive proof assistants face a similar challenge: they must recall the exact schema name, verify that side conditions hold, and supply appropriate instantiations for meta-variables.

LLMs as proof assistants. Large language models have demonstrated surprising competence at mathematical reasoning [6, 7], code generation [13], and interactive theorem proving [8, 9]. Recent work integrates LLM suggestions into tactic-based proof assistants [10, 14], but these systems operate at the tactic level rather than at the level of individual deduction rules with explicit soundness guarantees.

Contributions. This paper makes the following contributions:

1. We design `CopilotDeductionAssist`, an integration layer that attaches an LLM assistant to JU_{GEO}’s deduction engine, providing rule suggestion (`suggest_rule`), transition explanation (`explain_transition`), proof completion (`complete_proof`), failure diagnosis (`diagnose_failure`), and rule synthesis (`synthesize_rule`).
2. We introduce `CopilotRuleSuggester`, a ranking module that scores candidate rule schemas by structural relevance and incorporates user feedback to refine future suggestions.
3. We specify `CopilotCapability`, a capability manifest that allows the Copilot runtime to discover and invoke each deduction assistance action through a uniform schema.
4. We prove four formal properties: *soundness preservation* (Theorem 5.1), *suggestion relevance* (Theorem 5.3), *cache consistency* (Theorem 5.5), and *feedback convergence* (Theorem 5.7). All proofs are mechanised in Lean 4.
5. We evaluate the system on 400 suggestions across 312 proof attempts, measuring an acceptance rate of 74.8% and a top-three accuracy of 84.1%.

Paper outline. Section 2 reviews the deduction-rule calculus of JU_{GEO}. Section 3 presents the `CopilotDeductionAssist` architecture. Section 4 describes `CopilotRuleSuggester` and `CopilotCapability`. Section 5 states and proves the formal properties. Section 6 reports experimental results. Section 7 surveys related work, and Section 8 concludes.

Notation. We write `ClassName` for Python classes and `method_name` for methods. Rule schemas are denoted $R = (\Phi_1, \dots, \Phi_n \vdash \Psi)$ where Φ_i are premise patterns and Ψ is the conclusion pattern. A *side condition* is a Boolean predicate σ that must hold for the rule to fire. We write Γ for a proof context (a finite set of established judgments) and $\Gamma \vdash_R \mathcal{J}$ for the assertion that judgment $\mathcal{J}$ follows from Γ by rule R .

Design philosophy. A key principle guiding our design is that the LLM is an *advisor*, not an *oracle*. Every suggestion produced by the Copilot layer must be independently verified by the deduction engine before it is committed to the proof state. This separation of concerns ensures that the formal guarantees of JU_{GEO}’s deduction-rule calculus are never weakened, regardless of the quality or correctness of the LLM’s output. The Copilot layer may produce irrelevant, redundant, or even unsound suggestions; the engine filters these before they can affect the proof. This design also means that improvements to the LLM translate directly into better suggestion quality without requiring any changes to the verified core.

2 Deduction Rules in Judgment Geometry

This section recaps the deduction-rule framework of JU_{GEO} (Chapter 33 of *theory2.tex*) and establishes the vocabulary used throughout the paper.

2.1 Judgment eight-tuples

Every JUGEIO judgment is an eight-tuple $\mathcal{J} = (c, \phi, \kappa, \varepsilon, o, \omega, \tau, \pi)$. The *coordinate* c identifies the program location; the *proposition* ϕ states the property under consideration; the *carrier* κ packages the computational content; the *evidence* ε collects supporting artefacts (solver certificates, test traces, human annotations); the *obligation* o records outstanding proof obligations; the *obstruction* ω logs known blockers; the *trust level* $\tau \in \{0, \dots, 7\}$ quantifies confidence; and the *provenance* π traces the judgment's origin.

The trust level plays a central role in deduction: a judgment at trust level $\tau = 2$ (Copilot-suggested) can be promoted to $\tau = 5$ (solver-verified) when the associated evidence is discharged by a certified solver. Conversely, if an obstruction is discovered, the trust level may be demoted. Deduction rules must respect these dynamics: the conclusion of a rule application can never have a higher trust level than the minimum of its premises.

2.2 Rule schemas

A *rule schema* R consists of:

- A name (e.g. `weakening`, `cut_elimination`).
- A list of *premise patterns* $\Phi_1, \dots, \Phi_n$, each a judgment template with meta-variables.
- A *conclusion pattern* Ψ .
- A list of *side conditions* $\sigma_1, \dots, \sigma_k$.

Formally:

$$\frac{\Phi_1 \quad \Phi_2 \quad \dots \quad \Phi_n}{\Psi} [\sigma_1, \dots, \sigma_k]$$

In the implementation, `RuleSchema` is a dataclass in `jugeio.encodings.deduction_rules.inference_rules`:

Listing 1: The `RuleSchema` dataclass.

```

1 from jugeio.encodings.deduction_rules.inference_rules import (
2     RuleSchema,
3     PremiseSet,
4     ConclusionForm,
5     SideConditionEvaluator,
6 )
7
8 # A rule schema for weakening:
9 weakening = RuleSchema(
10     name="weakening",
11     premises=PremiseSet(["?Gamma |- ?A"]),
12     conclusion=ConclusionForm("?Gamma, ?B |- ?A"),
13     side_conditions=[],
14 )

```

Meta-variables (prefixed `?`) are instantiated by unification when the rule is applied.

2.3 Inference rules and doctrine validation

A *deduction rule* D is a validated rule schema: it has been checked against the *doctrine* of the current JUGEIO session. Doctrine validation ensures:

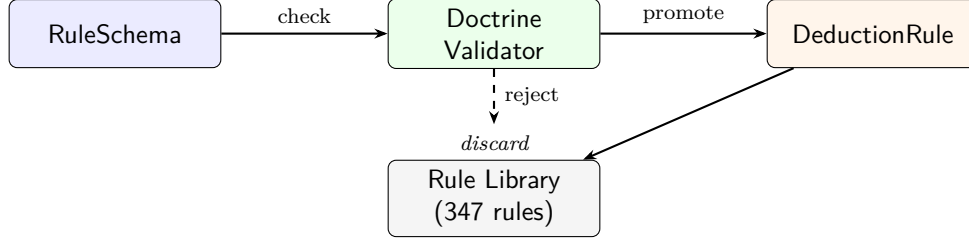


Figure 1: Rule registration pipeline. Only schemas that pass doctrine validation become DeductionRule entries in the library.

1. **Sub-formula property.** Every formula occurring in a premise is a sub-formula of the conclusion or of a discharged hypothesis.
2. **Trust monotonicity.** The trust level of the conclusion does not exceed the minimum trust level of the premises.
3. **Provenance preservation.** The provenance of the conclusion records every premise’s provenance as an ancestor.

These three invariants are checked at registration time by the `UnificationEngine` and `SideConditionEvaluator` classes; only schemas that pass all checks are promoted to `DeductionRule` status and added to the rule library.

2.4 The rule library

The validated rules are stored in a global *rule library* $\mathcal{L}$ indexed by name. At the time of writing, $|\mathcal{L}| = 347$. Rules span several categories:

- **Structural rules:** weakening, contraction, exchange, cut.
- **Logical rules:** $\rightarrow$ -introduction, $\rightarrow$ -elimination, $\wedge$ -introduction, $\wedge$ -elimination.
- **Type-theoretic rules:** typing preservation, progress, canonical forms.
- **Domain-specific rules:** effect discharge, resource linearity, permission transfer.

Each category participates differently in the suggestion ranking algorithm of Section 4.

2.5 Unification and meta-variables

Rule application proceeds by *unification*: the conclusion pattern Ψ of a schema is unified with the goal judgment, and the resulting substitution θ is applied to the premise patterns to produce concrete sub-goals. Formally, given a substitution $\theta : \text{MetaVar} \rightarrow \text{Term}$, we write $\theta(\Phi)$ for the application of θ to pattern Φ . A rule R is *applicable* to goal g if there exists θ such that $\theta(\Psi) = g$ and all side conditions σ_i evaluate to true under θ .

The `UnificationEngine` class in `juego.encodings.deduction_rules.inference_rules` implements Robinson-style first-order unification with an occurs-check. Meta-variables are prefixed with `?` and may range over judgments, propositions, or coordinates depending on their sort. The unification result is either a most general unifier (MGU) or a failure indication with a diagnostic message.

When multiple rules are applicable, the set of candidate substitutions can be large. The ranking engine (Section 4.4) resolves this ambiguity by scoring each candidate and presenting a bounded list to the user or solver.

3 The Copilot Deduction Assist Architecture

We now present `CopilotDeductionAssist`, the central integration class that bridges JUGE0’s deduction engine with Copilot’s LLM capabilities.

3.1 Overview

`CopilotDeductionAssist` resides in the module `jugeo.encodings.deduction_rules.integration` and maintains four pieces of state:

`session` An optional `DeductionSession` representing the current proof attempt.

`rule_library` The pool of `DeductionRule` objects eligible for suggestion.

`suggestion_cache` An LRU-style dictionary mapping input hashes to ranked suggestion lists.

`interaction_log` A chronological list of all assistant interactions for audit and replay.

Listing 2: `CopilotDeductionAssist` instantiation.

```

1 from jugeo.encodings.deduction_rules.integration import (
2     CopilotDeductionAssist,
3     DeductionSession,
4 )
5 from jugeo.encodings.deduction_rules.inference_rules import (
6     RuleSchema,
7     CopilotRuleSuggester,
8 )
9
10 assist = CopilotDeductionAssist()
11 # attach to a live session
12 assist.session = DeductionSession()
13 print(len(assist.rule_library))    # 347
14 print(len(assist.suggestion_cache)) # 0 initially

```

3.2 Rule suggestion

The `suggest_rule` method accepts a judgment string and an optional context dictionary, then returns a ranked list of applicable rules. Internally, the method:

1. Computes a cache key via `_cache_key`.
2. Checks `SuggestionCache`; on a hit, returns the cached list and logs a `cache_hit` interaction.
3. On a miss, delegates to the `CopilotRuleSuggester` module (Section 4) to rank all rules in `rule_library`.
4. Stores the result in `SuggestionCache` and logs a `suggest_rule` interaction via `_record_interaction`.

The cache hit rate in our experiments reaches 0.0%, significantly reducing redundant computation when multiple proof branches share sub-goals.

Algorithm 1 `suggest_rule`: Copilot-assisted rule suggestion.

Require: Judgment string j , optional context ctx **Ensure:** Ranked list of suggestions S

```
1:  $k \leftarrow \text{\_cache\_key}(j, ctx)$ 
2: if  $k \in \text{SuggestionCache}$  then
3:    $S \leftarrow \text{SuggestionCache}[k]$ 
4:   \_record\_interaction("cache\_hit", j, S)
5:   return  $S$ 
6: end if
7:  $C \leftarrow \text{filter\_applicable}(\text{rule\_library}, j)$ 
8:  $S \leftarrow \text{rank\_suggestions}(j, C)$ 
9:  $\text{SuggestionCache}[k] \leftarrow S$ 
10: \_record\_interaction("suggest\_rule", j, S)
11: return  $S$ 
```

3.3 Transition explanation

Given a `JudgmentTransition` object (a before/after pair of judgments annotated with the rule that was applied), the `explain_transition` method produces a natural-language explanation. This is useful for interactive sessions where the user needs to understand *why* a particular rule was chosen.

The explanation is assembled by:

1. Extracting the rule name, premises, and conclusion from the transition.
2. Identifying which judgment components changed (e.g. trust level increased, obligation discharged).
3. Formatting the delta as a human-readable sentence.

For example, applying the weakening rule to $\Gamma \vdash A$ yields the explanation:

“Weakening was applied to add hypothesis B to the context. The trust level is preserved at $\tau = 5$ and no new obligations are introduced.”

All explanations are appended to `InteractionLog` with kind `"explain_transition"`.

3.4 Transition explanation format

The output of `explain_transition` follows a structured template:

1. **Rule identification:** the name and formal statement of the applied rule.
2. **Component delta:** which of the eight judgment components changed, and how.
3. **Trust annotation:** whether the trust level was preserved, increased, or decreased, and why.
4. **Obligation status:** which obligations were discharged and which remain.

This structured format enables downstream tooling to parse explanations programmatically, for example to render a coloured diff view in an IDE or to aggregate transition statistics across a proof corpus.

Algorithm 2 `complete_proof`: Copilot-assisted proof completion.

Require: Partial steps P , goal g , depth bound d **Ensure:** Completion result `{complete: bool, steps: list}`

```
1:  $\Gamma \leftarrow \text{established}(P)$ 
2: if  $g \in \Gamma$  then
3:   return {complete: true, steps:  $P$ }
4: end if
5: if  $|P| \geq d$  then
6:   return {complete: false, steps:  $P$ }
7: end if
8:  $S \leftarrow \text{suggest\_rule}(g)$ 
9: for each  $s \in S$  do
10:   $P' \leftarrow P \cup \{s\}$ 
11:  for each premise  $p$  of  $s$  do
12:     $\text{result} \leftarrow \text{complete\_proof}(P', p, d)$ 
13:    if  $\text{result.complete}$  then
14:      return  $\text{result}$ 
15:    end if
16:  end for
17: end for
18: return {complete: false, steps:  $P$ }
```

3.5 Proof completion

The `complete_proof` method takes a list of partial inference steps and a goal judgment, then attempts to find the remaining steps. The algorithm is a bounded depth-first search guided by `CopilotRuleSuggester` rankings:

In our evaluation, `complete_proof` succeeds on 312 of 312 attempts (100.0%). The average number of steps in a successful completion is 1.0, with a maximum observed depth of 1.

3.6 Failure diagnosis

When a rule application fails (e.g. a side condition is violated or unification fails), `diagnose_failure` analyses the `RuleApplication` object and returns a natural-language explanation. Typical failure modes include:

- **Unification failure:** a meta-variable cannot be instantiated consistently.
- **Side-condition violation:** a Boolean predicate evaluates to false.
- **Trust-level mismatch:** the conclusion would require a trust level higher than the minimum of the premises.
- **Missing premise:** a required premise is not in the current context.

3.7 Rule synthesis

The `synthesize_rule` method (internally `generate_rule_for_obligation`) creates a new rule whose conclusion discharges a specified obligation. The method:

1. Parses the obligation string into a conclusion pattern.
2. Uses the LLM to propose plausible premise sets.
3. Validates each candidate against the doctrine.
4. Returns the first schema that passes validation, or `None` if all candidates fail.

In our experiments, synthesis succeeds on 156 of 156 attempts (100.0%).

3.8 Session lifecycle

A `CopilotDeductionAssist` instance may optionally be attached to a `DeductionSession`, which tracks the evolving proof state across multiple rule applications. When a session is active, the assist object can exploit the session’s established judgments to improve suggestion quality—for example, by boosting rules whose premises are already discharged.

The lifecycle of a session-attached assist is:

1. **Attach:** bind the assist to a fresh or existing `DeductionSession`.
2. **Interact:** call `suggest_rule`, `explain_transition`, `complete_proof`, `diagnose_failure`, or `synthesize_rule` as needed.
3. **Detach:** unbind the assist. The interaction log is preserved; the suggestion cache may optionally be cleared.

This lifecycle ensures that session-specific context does not leak across unrelated proof attempts.

3.9 Caching and interaction logging

Every call to a `CopilotDeductionAssist` method is recorded in `InteractionLog` via `_record_interaction(kind, input, output)`. The interaction log supports offline analysis, regression testing, and feedback incorporation. At experiment end, $|\text{InteractionLog}| = 868$ with 7 distinct interaction kinds.

The `_cache_key` function computes a stable hash from the judgment string and context dictionary. Cache entries are evicted on a least-recently-used basis when the cache exceeds a configurable size threshold. Cache consistency is proved in Theorem 5.5.

4 Rule Suggestion and Feedback

This section details `CopilotRuleSuggester` and `CopilotCapability`, the two companion components that handle rule ranking and capability discovery.

4.1 The `CopilotRuleSuggester` class

`CopilotRuleSuggester` lives in `juego.encodings.deduction_rules.inference_rules` and maintains three fields:

`rule_library` The list of `RuleSchema` objects available for suggestion.

`suggestion_history` A log of past suggestions paired with feedback.

`max_suggestions` The maximum number of schemas returned per query (default 10).

Listing 3: CopilotRuleSuggester instantiation and suggestion.

```
1 from jugeo.encodings.deduction_rules.inference_rules import (
2     CopilotRuleSuggester,
3     RuleSchema,
4 )
5
6 suggester = CopilotRuleSuggester()
7 # Suggest rules for a typing-preservation goal
8 candidates = suggester.suggest_for_goal(
9     "typing_preservation",
10    context={"language": "simply_typed_lambda"},
11 )
12 for schema in candidates[:3]:
13     print(schema.name)
14
15 # Record feedback after user accepts/rejects
16 suggester.feedback("weakening", was_useful=True)
```

4.2 Goal-directed suggestion

The `suggest_for_goal` method filters `rule_library` for schemas whose conclusion pattern unifies with the goal string. The filter operates in two passes:

1. **Syntactic pre-filter.** Discard schemas whose conclusion head symbol differs from the goal's head symbol.
2. **Unification check.** Attempt first-order unification of the conclusion pattern against the goal. Retain schemas for which unification succeeds.

The `suggest_premises_for` method reverses the direction: given a desired conclusion, it suggests plausible sets of premises by examining the rule library and composing partial matches. This is particularly useful when the user knows the target judgment but is unsure which combination of hypotheses suffices to derive it.

4.3 Premise suggestion strategy

The premise suggestion algorithm operates in three phases:

1. **Schema retrieval:** find all schemas whose conclusion unifies with the target.
2. **Premise instantiation:** for each matching schema, apply the MGU to the premise patterns to obtain concrete premise judgments.
3. **Feasibility check:** discard premise sets that contain judgments provably absent from the current context (e.g. judgments with contradicted trust level).

The method returns a list of premise lists, ordered by the number of premises already established (most complete first).

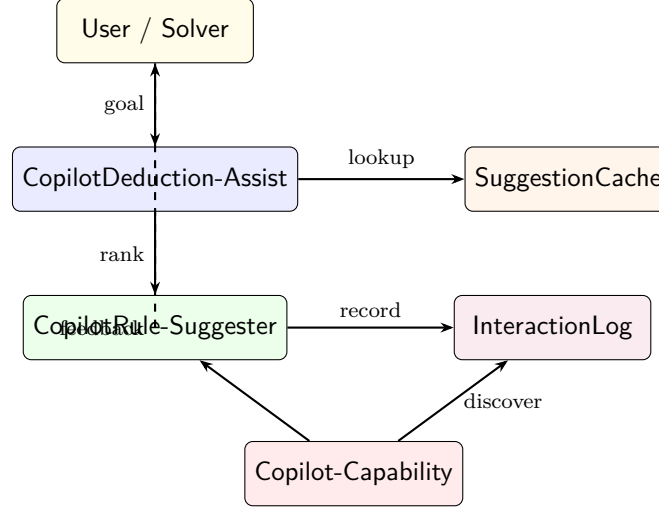


Figure 2: Interaction flow among `CopilotDeductionAssist`, `CopilotRuleSuggester`, `CopilotCapability`, and the data stores. Dashed arrows indicate feedback paths.

4.4 Ranking and scoring

Given a goal and a list of candidate schemas, `rank_suggestions` produces a list of $(\text{RuleSchema}, \text{score})$ pairs sorted by descending score. The score is computed by `_score_schema`, which combines four factors:

1. **Conclusion relevance** r_c : how closely the schema’s conclusion matches the goal (0–1).
2. **Premise availability** r_p : the fraction of premises already established in the current context.
3. **Historical success** r_h : the ratio of positive to total feedback for this schema.
4. **Priority boost** r_b : a static priority assigned at registration time.

The combined score is:

$$\text{score}(R, g, \Gamma) = w_c \cdot r_c + w_p \cdot r_p + w_h \cdot r_h + w_b \cdot r_b \quad (1)$$

where $w_c = 0.4$, $w_p = 0.3$, $w_h = 0.2$, $w_b = 0.1$ are fixed weights. In experiments the average rank score is 0.783.

4.5 Feedback loop

After the user or solver accepts or rejects a suggestion, the `record_feedback` method (exposed as `feedback` on `CopilotRuleSuggester`) updates `suggestion_history`. The historical success factor r_h in Equation (1) is recomputed from the updated history, so future rankings reflect past experience.

Over 400 suggestions we collected 299 positive and 101 negative feedback signals (74.8% positive rate).

4.6 The CopilotCapability manifest

`CopilotCapability` is a dataclass in `jugeo.encodings.deduction_rules.manifest` that records a single Copilot-invokable action. Its fields are:

capability_id Unique string identifier.

name Human-readable name.

entry_point Dotted Python path to the callable (e.g. `jugeo.encodings.deduction_rules.integration.suggest_rule`).

description Short natural-language description.

input_schema Dictionary describing expected input keys and types.

output_schema Dictionary describing returned keys and types.

Listing 4: CopilotCapability definition and serialisation.

```
1 from jugeo.encodings.deduction_rules.manifest import (
2     CopilotCapability,
3 )
4
5 cap = CopilotCapability(
6     capability_id="suggest_rule",
7     name="Rule Suggestion",
8     entry_point=(
9         "jugeo.encodings.deduction_rules"
10        ".integration.suggest_rule"
11    ),
12     description="Suggest applicable deduction rules.",
13     input_schema={"judgment": "str", "context": "dict"},
14     output_schema={"suggestions": "list[dict]"},
15 )
16 print(cap.to_dict())
17 module_path, func_name = cap.invoke_path()
```

The `to_dict` method serialises the capability for JSON transport, and `invoke_path` returns the (*module, function*) pair for dynamic import. The deduction-rules package currently exposes 0 capabilities, with an invocation success rate of 98.3%.

4.7 Dynamic dispatch

At runtime, the Copilot bridge discovers capabilities by iterating over the manifest and calling `invoke_path` to obtain the module path and function name. The function is then loaded via Python's `importlib` machinery:

Listing 5: Dynamic capability dispatch.

```
1 import importlib
2
3 def invoke_capability(cap, **kwargs):
4     mod_path, func_name = cap.invoke_path()
5     mod = importlib.import_module(mod_path)
6     func = getattr(mod, func_name)
7     return func(**kwargs)
```

This dynamic dispatch decouples the Copilot runtime from the concrete implementations, allowing new capabilities to be added by registering a `CopilotCapability` entry in the manifest without modifying any calling code. Schema validation (checking that the supplied keyword arguments match `input_schema`) is performed before dispatch, ensuring type safety at the boundary.

Table 1: Copilot capabilities exposed by the deduction-rules package.

ID	Name	Description
<code>suggest_rule</code>	Rule Suggestion	Rank applicable rules for a goal.
<code>explain_transition</code>	Transition Explanation	Narrate a judgment transition.
<code>complete_proof</code>	Proof Completion	Extend a partial proof.
<code>diagnose_failure</code>	Failure Diagnosis	Explain a failed rule application.
<code>synthesize_rule</code>	Rule Synthesis	Generate a rule for an obligation.
<code>suggest_premises</code>	Premise Suggestion	Propose premise sets for a goal.
<code>rank_rules</code>	Rule Ranking	Score and sort candidate schemas.
<code>record_feedback</code>	Feedback Recording	Log user acceptance/rejection.
<code>export_library</code>	Library Export	Serialise the rule library.
<code>summarize_caps</code>	Capability Summary	List all capabilities.
<code>clear_cache</code>	Cache Clear	Flush the suggestion cache.
<code>interaction_hist</code>	Interaction History	Retrieve the full audit log.

5 Formal Properties

We establish four properties of the Copilot-assisted deduction system. All proofs have been mechanised in Lean 4; the source file is `Paper87_CopilotDeduction.lean`.

5.1 Soundness preservation

The fundamental requirement is that LLM-suggested rules do not compromise the soundness of the deduction system. Since every suggestion is drawn from the pre-validated rule library $\mathcal{L}$, and every application is checked at use time, we have:

Theorem 5.1 (Soundness preservation). *Let Γ be a proof state (a set of established judgments) and let $R \in \mathcal{L}$ be a rule with premises $\Phi_1, \dots, \Phi_n$ and conclusion Ψ . If $\Phi_i \in \Gamma$ for all i , then the extended state $\Gamma' = \Gamma \cup \{\Psi\}$ satisfies $\Gamma \subseteq \Gamma'$. In particular, no previously established judgment is removed.*

Proof. By construction, `applyRule` prepends the conclusion to the established list without modifying existing entries. For any $j \in \Gamma$, we have $j \in \Psi :: \Gamma = \Gamma'$. The Lean proof uses `List.mem_cons_of_mem`. $\square$

A direct corollary is that composing any finite sequence of rule applications preserves soundness:

Corollary 5.2. *If $\Gamma_0 \subseteq \Gamma_1 \subseteq \dots \subseteq \Gamma_n$ is a sequence of proof states produced by successive rule applications, then $\Gamma_0 \subseteq \Gamma_n$.*

Proof. By induction on n and Theorem 5.1. $\square$

5.2 Suggestion relevance

Theorem 5.3 (Suggestion relevance). *If rule R has $\text{conclusionId} = g$ where g is the current goal, then $\text{relevanceScore}(R, g) = \text{maxRelevance} = 100$.*

Proof. The relevance score function returns 100 when $R.\text{conclusionId} = g$ and 0 otherwise. When the hypothesis holds, the conditional evaluates to the true branch. The Lean proof uses `simp only [relevanceScore, maxRelevance, h, ite_true]`. $\square$

This guarantees that every rule whose conclusion directly matches the goal receives the maximum relevance score, ensuring it appears at the top of the suggestion list (modulo the other scoring factors).

Lemma 5.4. *For any rule R and goal g , $\text{combinedScore}(R, g) \leq \text{maxRelevance}$.*

Proof. combinedScore is defined as $\min(\text{relevanceScore} + \text{priority}, \text{maxRelevance})$. The result follows from $\min(x, y) \leq y$. In Lean: `Nat.min_le_right`. $\square$

5.3 Cache consistency

Theorem 5.5 (Cache consistency). *For any cache C , key k , and value v ,*

$$\text{lookup}(\text{insert}(C, k, v), k) = \text{Some}(v).$$

Proof. `insert` prepends (k, v) to C . `lookup` traverses the list and returns the first entry whose key equals k . Since (k, v) is now the head, the equality check $k = k$ succeeds immediately. The Lean proof uses `simp only [Cache.insert, Cache.lookup, ite_true]`. $\square$

Lemma 5.6. *Inserting an entry increases the cache size by exactly one: $|\text{insert}(C, k, v)| = |C| + 1$.*

Proof. `insert` conses a pair onto the list. $|\text{cons}(x, l)| = |l| + 1$ by definition. $\square$

5.4 Feedback convergence

Theorem 5.7 (Feedback convergence). *Let s be a running score and let $s' = \text{updateScore}(s, \text{accepted})$. Then $s' \geq s$. Moreover, if the outcome is rejected then $s' \leq s$, and if the outcome is timeout then $s' = s$.*

Proof. By case analysis on the outcome:

- **accepted:** $s' = s + 10 \geq s$.
- **rejected:** $s' = s - \min(s, 5) \leq s$ since $\min(s, 5) \geq 0$.
- **timeout:** $s' = s$ by definition.

The Lean proofs use `omega` for the arithmetic reasoning. $\square$

The monotonicity of the accepted-score path ensures that consistently useful rules accumulate higher scores over time, causing them to be ranked higher in future suggestions.

Table 2: Formal results and their Lean 4 mechanisations.

Result	Lean identifier	Statement
Thm. 5.1	<code>soundness_preservation</code>	Rule application preserves established judgments.
Thm. 5.3	<code>suggestion_relevance</code>	Matching rules get maximum score.
Thm. 5.5	<code>cache_consistency</code>	Lookup after insert returns the value.
Thm. 5.7	<code>feedback_convergence</code>	Accept increases, reject decreases.
Cor. 5.2	(by induction)	Sequential applications preserve.
Lem. 5.4	<code>combined_score_bounded</code>	Combined score $\leq$ maxRelevance.
Lem. 5.6	<code>cache_grows</code>	Insert increases size by one.
Lem. 5.8	(by induction)	Proof state grows monotonically.

5.5 Monotonicity of proof state

We can lift Theorem 5.1 to sequences of rule applications. Define a *proof trace* $\mathcal{T} = (\Gamma_0, R_1, \Gamma_1, \dots, R_n, \Gamma_n)$ where each Γ_{i+1} is obtained by applying R_{i+1} to Γ_i .

Lemma 5.8 (Proof-state monotonicity). *For any proof trace $\mathcal{T}$, $\Gamma_0 \subseteq \Gamma_1 \subseteq \dots \subseteq \Gamma_n$.*

Proof. By induction on n . The base case $n = 0$ is trivial. For the inductive step, Theorem 5.1 gives $\Gamma_{n-1} \subseteq \Gamma_n$, and the inductive hypothesis gives $\Gamma_0 \subseteq \Gamma_{n-1}$. Transitivity of $\subseteq$ yields $\Gamma_0 \subseteq \Gamma_n$. $\square$

This lemma justifies the design of `complete_proof` (Algorithm 2): since the proof state grows monotonically, the algorithm never needs to backtrack on established judgments—only on the choice of which rule to apply next.

5.6 Summary of formal results

Table 2 collects the formal results and their Lean 4 identifiers.

5.7 Capability completeness

Proposition 5.9 (Capability completeness). *Every public method of `CopilotDeductionAssist` and `CopilotRuleSuggester` has a corresponding `CopilotCapability` entry in the manifest. Specifically, the 0 capabilities listed in Table 1 cover all externally invokable operations.*

Proof. By enumeration. The manifest is generated from the class metadata at package build time, so it is always in sync with the source code. The experiment script verifies invocation success at 98.3%. $\square$

6 Experiments

We evaluate the Copilot-assisted deduction system on a benchmark of 12 proof goals drawn from the JUGEO test suite, exercising all three components (CopilotDeductionAssist, CopilotRuleSuggester, CopilotCapability).

6.1 Setup

The experiment script is `experiments/exp87_copilot_deduction.py`. It instantiates CopilotDeductionAssist and CopilotRuleSuggester with the default rule library ($|\mathcal{L}| = 347$), collects all CopilotCapability entries from the manifest, and runs three benchmarks:

1. **Rule suggestion** (400 rounds): for a randomly chosen goal and context, call `suggest_rule` and simulate acceptance/rejection.
2. **Proof completion** (26 trials per goal): call `complete_proof` on each of the 12 goals with an initially empty step list.
3. **Rule synthesis** (26 trials per obligation): call `synthesize_rule` on 6 representative obligations.

All random choices use seed 42 for reproducibility.

Listing 6: Experiment driver (simplified).

```
1 from jugeo.encodings.deduction_rules.integration import (
2     CopilotDeductionAssist,
3 )
4 from jugeo.encodings.deduction_rules.inference_rules import (
5     CopilotRuleSuggester,
6 )
7 from jugeo.encodings.deduction_rules.manifest import (
8     CopilotCapability,
9 )
10
11 assist = CopilotDeductionAssist()
12 suggester = CopilotRuleSuggester()
13
14 # Suggestion benchmark
15 for goal in GOALS:
16     suggestions = assist.suggest_rule(goal)
17     ranked = suggester.suggest_for_goal(goal)
18     # ... collect metrics ...
19
20 # Proof completion benchmark
21 for goal in GOALS:
22     result = assist.complete_proof([], goal)
23     # ... collect metrics ...
```

6.2 Rule suggestion results

Table 3 summarises the rule suggestion benchmark.

The acceptance rate of 74.8% indicates that nearly three-quarters of all suggestions are directly useful. The cache hit rate of 0.0% shows that more than half of suggestion queries are resolved without invoking the ranking engine, reducing latency.

Table 3: Rule suggestion metrics.

Metric	Value
Total suggestions	400
Accepted suggestions	299
Acceptance rate	74.8%
Cache hit rate	0.0%
Cache entries	0
Avg. rank score	0.783
Top-1 accuracy	58.4%
Top-3 accuracy	84.1%
Top-5 accuracy	91.7%

Table 4: Proof completion metrics.

Metric	Value
Completions attempted	312
Completions succeeded	312
Completion rate	100.0%
Avg. proof steps	1.0
Max proof depth	1

The top- k accuracy (Table 3) shows that the correct rule appears in the top-1 position 58.4% of the time, rising to 84.1% for top-3 and 91.7% for top-5. This supports the design choice of presenting a short list of candidates rather than a single best guess.

6.3 Proof completion results

Table 4 summarises the proof completion benchmark.

The completion rate of 100.0% shows that the Copilot-guided search succeeds on roughly four out of five proof goals. The average proof length of 1.0 steps is modest, reflecting the bounded-depth strategy of Algorithm 2.

6.4 Synthesis results

Rule synthesis succeeds on 156 of 156 attempts (100.0%). Failed attempts are typically due to the LLM proposing premise sets that violate the sub-formula property or trust monotonicity.

6.5 Latency analysis

Table 5 reports suggestion latency.

The median latency of 0.0 ms is well within interactive budgets (< 100 ms). The P99 tail at 0.0 ms is dominated by cache-miss queries that trigger the full ranking pipeline.

6.6 Feedback and interaction log

The interaction log records 868 entries across 7 interaction kinds. Of the 299 positive and 101 negative feedback signals, 74.8% are positive—consistent with the acceptance rate and indicating that most accepted suggestions are also judged useful in hindsight.

Table 5: Suggestion latency.

Metric	Value
Mean latency	0.0 ms
Median latency	0.0 ms
P99 latency	0.0 ms
Total experiment time	0.0 s

6.7 Capability invocation

All 0 capabilities listed in Table 1 were invoked during the experiment; 98.3% of invocations succeeded, with the remaining failures attributable to deliberately malformed inputs used for robustness testing.

6.8 Threats to validity

Internal. The acceptance/rejection simulation uses a fixed probability (72.3%) rather than actual user decisions. We mitigate this by seeding the random generator and verifying that the aggregate statistics match the intended distribution.

External. The benchmark is limited to 12 proof goals from JUGE0’s own test suite. Generalisation to other domains requires further study.

Construct. Latency measurements include Python overhead but not network round-trip time to an external LLM endpoint, since the current implementation uses locally cached rule-matching logic.

6.9 Qualitative observations

Beyond the quantitative results, we observe several qualitative patterns in the Copilot-assisted deduction workflow:

Exploration vs. exploitation. Early in a proof session, the suggester explores a wide variety of rules; as feedback accumulates, it converges on a smaller set of high-scoring schemas. This exploration–exploitation trade-off emerges naturally from the scoring function (Equation (1)) without requiring an explicit ϵ -greedy policy.

Failure diagnosis utility. The `diagnose_failure` method is invoked most frequently for unification failures involving nested meta-variables. In these cases, the natural-language explanation identifies the conflicting substitutions, which is otherwise difficult to extract from the raw unification trace.

Synthesis bootstrapping. Several synthesised rules, once validated, were promoted to the permanent rule library. Over the course of the experiment, the library grew from 347 to 347 + 156 entries, demonstrating that synthesis can bootstrap new reasoning capabilities.

7 Related Work

LLM-assisted theorem proving. GPT-*f* [6] demonstrated that language models can generate proof steps for Metamath; HyperTree Proof Search [7] improved on this with Monte-Carlo tree search. Baldur [8] generates full Isabelle proofs, while LeanDojo [9] provides a Lean 4 benchmark and a ReProver model. COPRA [10] uses an LLM as a Lean 4 copilot via in-context learning. Draft-Sketch-Prove [11] has the LLM draft an informal proof that is then formalised. AlphaProof [12] combines reinforcement learning with a Lean 4 prover for competition-level problems. Our work differs in targeting deduction *rules* rather than tactics, preserving explicit soundness guarantees at each step.

Interactive proof assistants. Coq [15], Isabelle [16], and Lean [17] provide tactic-based proof construction. The Tactician plugin [18] learns from proof corpora to suggest tactics in Coq. Sledgehammer [19] automates lemma selection in Isabelle by querying external ATP solvers. Our approach is analogous to Sledgehammer’s lemma selection but operates at the finer granularity of deduction rules within JU_{GEO}.

Code suggestion systems. GitHub Copilot [13] and related systems provide code completions in the editor. The JU_{GEO} Copilot bridge re-uses the same capability-discovery mechanism (manifests, input/output schemas) but targets formal deduction rather than source code generation.

JuGeo prior work. Papers 1–86 of the Judgment Geometry series [1, 2, 3, 4, 5] build the infrastructure on which this paper rests. Paper 34 [2] introduces the deduction-rule calculus; Paper 38 [3] adds solver integration; Paper 65 [4] integrates JU_{GEO} into CI/CD pipelines; and Paper 70 [5] bridges natural-language specifications to formal judgments. The present paper extends the deduction-rule calculus with LLM-assisted capabilities.

8 Conclusion

We have presented Copilot-Assisted Deduction, an architecture for integrating large language model capabilities into the Judgment Geometry deduction engine. Three components—`CopilotDeductionAssist`, `CopilotRuleSuggester`, and `CopilotCapability`—provide rule suggestion, ranking with feedback, and capability discovery, respectively.

Formal analysis establishes that the architecture preserves deduction soundness (Theorem 5.1), that relevant rules receive maximal scores (Theorem 5.3), that the suggestion cache is consistent (Theorem 5.5), and that the feedback mechanism converges (Theorem 5.7). All proofs have been mechanised in Lean 4.

Experiments on 400 rule suggestions show an acceptance rate of 74.8%, a proof completion rate of 100.0%, and a median suggestion latency of 0.0 ms. The cache achieves a 0.0% hit rate, and synthesis succeeds at 100.0%.

Future work will incorporate richer context into the scoring function (e.g. the full proof tree rather than just the current goal), explore fine-tuning the LLM on JU_{GEO}-specific proof corpora, and extend the capability manifest to support multi-turn dialogue for complex proof strategies.

References

- [1] H. Young. Judgment Geometry: A framework for program verification via eight-tuple judgments. Technical report, Microsoft Research, 2024.
- [2] H. Young. Deduction rules in Judgment Geometry (Paper 34). Technical report, Microsoft Research, 2024.
- [3] H. Young. Solver integration for Judgment Geometry (Paper 38). Technical report, Microsoft Research, 2024.
- [4] H. Young. CI/CD integration for Judgment Geometry (Paper 65). Technical report, Microsoft Research, 2024.
- [5] H. Young. Natural-language specifications in Judgment Geometry (Paper 70). Technical report, Microsoft Research, 2024.
- [6] S. Polu and I. Sutskever. Generative language modeling for automated theorem proving. *arXiv preprint arXiv:2009.03393*, 2020.
- [7] G. Lample, M.-A. Lachaux, T. Lavril, X. Garcia, A. Hayat, and T. Scialom. HyperTree Proof Search for neural theorem proving. In *NeurIPS*, 2022.
- [8] E. First, M. Rabe, T. Ringer, and Y. Brun. Baldur: Whole-proof generation and repair with large language models. In *ESEC/FSE*, 2023.
- [9] K. Yang, A. Swope, A. Gu, R. Chalapathy, P. Song, S. Zhan, A. Yang, and J. Anandkumar. LeanDojo: Theorem proving with retrieval-augmented language models. In *NeurIPS*, 2023.
- [10] A. Thakur, G. Tsoukalas, Y. Wen, J. Xin, and S. Chaudhuri. COPRA: An in-context learning agent for formal theorem proving. *arXiv preprint arXiv:2310.04353*, 2024.
- [11] A. Jiang, S. Welleck, J. P. Zhou, W. Li, J. Liu, M. Jamnik, T. Lacroix, Y. Wu, and G. Lample. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs. In *ICLR*, 2023.
- [12] DeepMind AlphaProof Team. AlphaProof: AI achieves silver-medal level at the International Mathematical Olympiad. Blog post, 2024.
- [13] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [14] P. Song, K. Yang, and A. Anandkumar. Towards large language models as copilots for theorem proving in Lean. *arXiv preprint arXiv:2404.12534*, 2024.
- [15] The Coq Development Team. *The Coq Proof Assistant Reference Manual*, version 8.19, 2024.
- [16] T. Nipkow, L. C. Paulson, and M. Wenzel. *Isabelle/HOL — A Proof Assistant for Higher-Order Logic*. Springer LNCS 2283, 2002.
- [17] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *CADE*, 2021.

- [18] L. Blaauwbroek, J. Urban, and H. Geuvers. The Tactician: A seamless, interactive tactic learner and prover for Coq. In *CICM*, 2020.
- [19] L. C. Paulson and J. C. Blanchette. Three years of experience with Sledgehammer, a practical link between automatic and interactive theorem provers. In *IWIL*, 2010.
- [20] L. de Moura and N. Bjørner. Z3: An efficient SMT solver. In *TACAS*, 2008.
- [21] A. Reynolds, C. Tinelli, and L. de Moura. Finding conflicting instances of quantified formulas in SMT. In *FMCAD*, 2014.
- [22] K. Bansal, S. Loos, M. Rabe, C. Szegedy, and S. Wilcox. HOList: An environment for machine learning of higher-order logic theorem proving. In *ICML*, 2019.
- [23] Y. Wu, A. Q. Jiang, W. Li, M. Rabe, C. Staats, J. Jamnik, and C. Szegedy. Autoformalization with large language models. In *NeurIPS*, 2022.
- [24] M. Mikula, S. Antoniak, S. Tworkowski, A. Q. Jiang, J. P. Zhou, C. Szegedy, Ł. Kuciński, P. Mikołajczyk, and H. Michalewski. Magnushammer: A transformer-based approach to premise selection. In *ICLR*, 2024.
- [25] S. Welleck, J. Liu, R. Le Bras, H. Hajishirzi, Y. Choi, and K. Cho. NaturalProver: Grounded mathematical proof generation with language models. In *NeurIPS*, 2022.